

Program vs Process vs Thread

A Complete Visual Guide — with Diagrams — for Operating System Interview Prep

1. What is a Program?

A **program** is a passive entity — a set of instructions written by a programmer, stored as a file on disk (e.g. **main.exe**, **a.out**). It contains code and static data, but it is not running and does not consume CPU, memory, or any system resources on its own. Think of a program like a recipe written on paper: it describes what to do, but nothing happens until someone actually starts cooking.

Key characteristics of a program:

- **Static** — it just sits on the disk/storage as a file.
- Contains machine instructions + static data, produced by a compiler/linker from source code.
- Has no execution state (no program counter value, no registers in use, no memory allocated at runtime).
- One program can be used to start many processes (e.g. opening the same app twice).

2. What is a Process?

A **process** is a program **in execution** — an active entity. The moment you run a program, the OS loads it into memory, allocates resources, and creates a Process Control Block (PCB) to track it. From that point, it is a process: it has a unique Process ID (PID), its own memory address space, CPU register values, open file handles, and a current execution state.

A process's memory is divided into four main segments:

- **Text/Code segment** — the compiled instructions being executed (read-only).
- **Data segment** — global and static variables.
- **Heap** — dynamically allocated memory (malloc/new), grows upward.
- **Stack** — function calls, local variables, return addresses, grows downward.

Each process is **isolated** — it cannot directly access another process's memory. This isolation is what makes one crashing process not usually crash the entire system.

3. What is a Thread?

A **thread** is the smallest unit of CPU execution within a process — often called a "lightweight process." A single process can have one or many threads. All threads that belong to the same process **share** the same code, data, and heap segments and open files, but each thread keeps its **own** stack, CPU registers, program counter, and thread ID — so each can execute independently.

Why use multiple threads?

- **Concurrency** — different parts of an app can run at the same time (e.g. UI stays responsive while a file downloads).
- **Resource sharing** — threads share memory, so communication between them is faster/cheaper than between processes.
- **Cheaper creation & context switching** — no need to duplicate the entire address space like a new process would.
- **Better use of multi-core CPUs** — threads can genuinely run in parallel on separate cores.

Program → Process → Thread: Visual Comparison

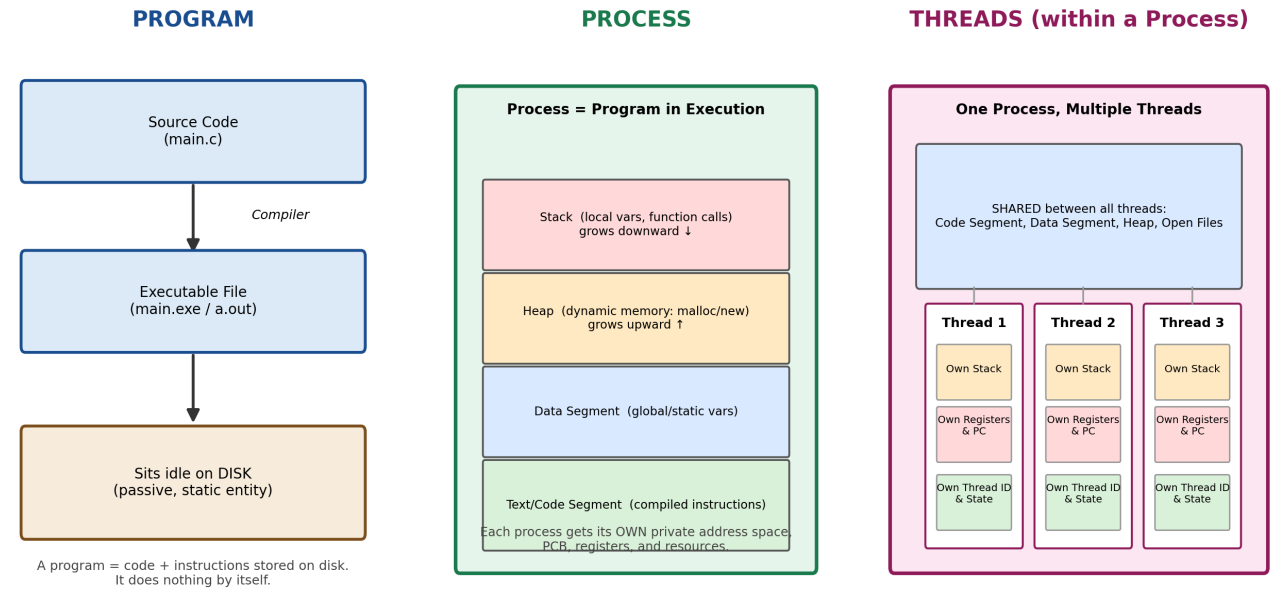


Figure 1: A program is static code on disk. Once run, it becomes a process with its own memory layout. A process can further split its work into multiple threads that share memory but keep their own stack and registers.

Quick Comparison Table

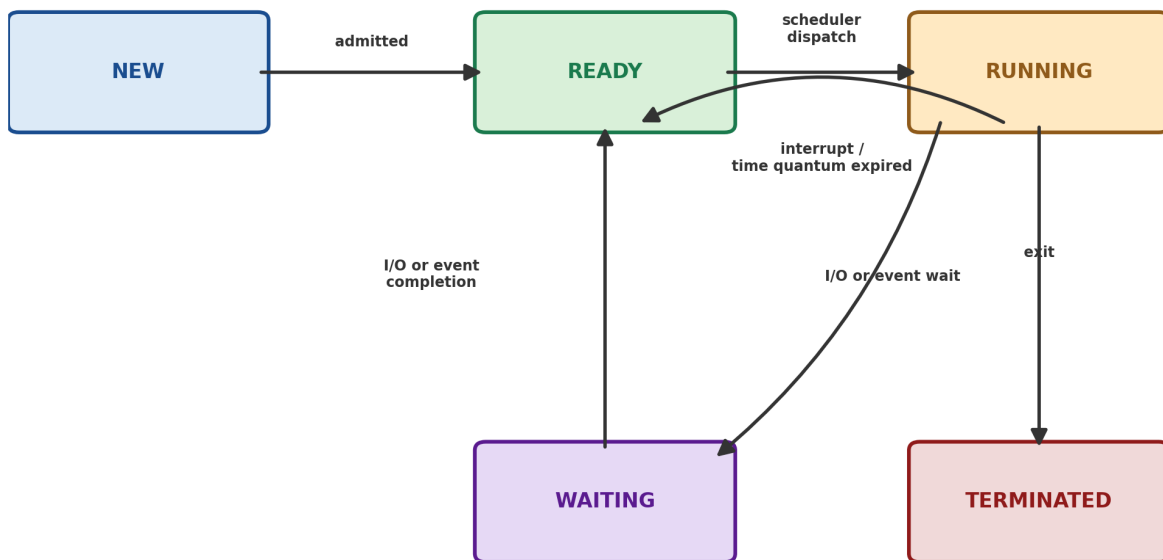
Aspect	Program	Process	Thread
Nature	Passive (static)	Active (dynamic)	Active (dynamic)
Location	Stored on disk	Loaded in memory	Exists inside a process
Memory	No runtime memory	Own isolated address space	Shares process's memory, own stack
Resources	None	Owens CPU time, memory, files, PCB	Shares parent process's resources
Creation cost	N/A	Heavy (fork/CreateProcess)	Lightweight (cheap to create/switch)
Communication	N/A	Needs IPC (pipes, sockets, shared mem)	Direct — shared memory, no IPC needed
Crash impact	N/A	Usually isolated to that process	Can crash the entire parent process

4. Process Lifecycle: States a Process Goes Through

Throughout its life, every process moves between five key states, managed entirely by the OS scheduler:

- **New** — the process is being created.
- **Ready** — the process is loaded in memory and waiting for the CPU to be assigned to it.
- **Running** — the process's instructions are actively being executed by the CPU.
- **Waiting (Blocked)** — the process is paused, waiting for an I/O operation or event to complete.
- **Terminated** — the process has finished execution and is being removed by the OS.

Process State Transition Diagram



Every process moves through these states during its lifetime, managed entirely by the OS scheduler.

Figure 2: Process State Transition Diagram — showing how a process moves between New, Ready, Running, Waiting, and Terminated states.

5. Full Journey: From Running a Program to Full Execution

Here is exactly what happens, step by step, from the moment you launch a program until the resulting process finishes execution and is removed from the system:

From Program to Full Execution: How a Process is Born, Runs, and Dies

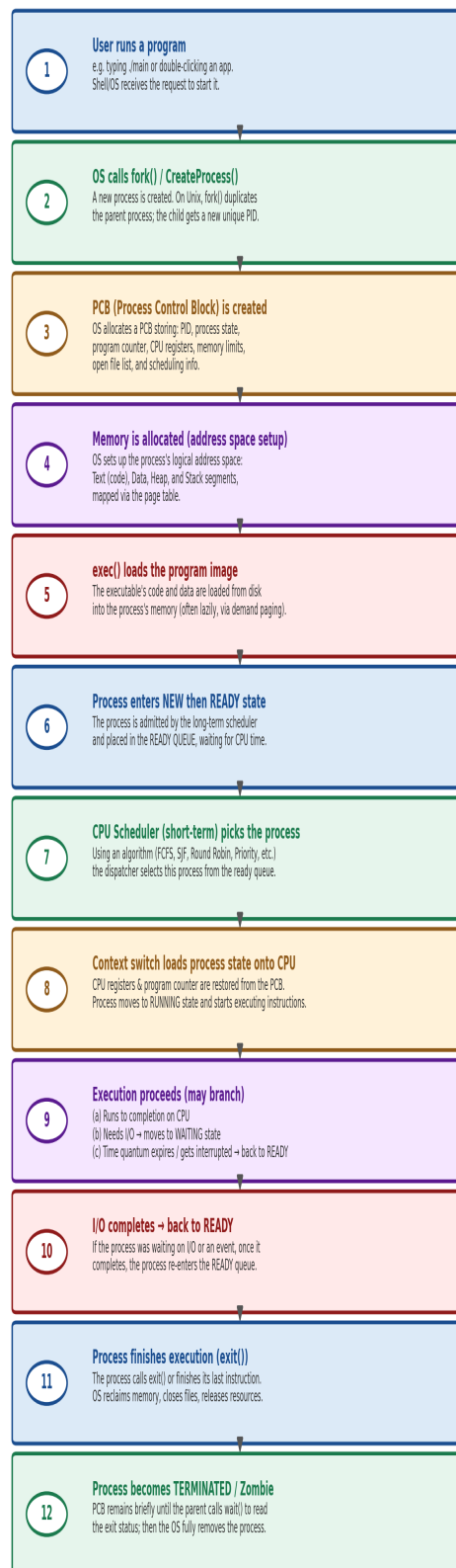


Figure 3: The complete flow of process creation, scheduling, execution, and termination.

6. Interview-Ready Summary

If asked to explain this live, this one-liner sequence covers it well:

“A **program** is passive code on disk. When executed, the OS creates a **process** — allocating memory, a PCB, and a PID — which moves through New → Ready → Running → (Waiting) → Terminated states as the scheduler manages CPU time. A process can also spawn multiple **threads**, which share the process's memory and resources but each execute independently with their own stack and registers, enabling concurrency within a single process.”

Prepared for Welldev OS interview preparation.